Лабораторная работа № 2

Класс Three: теория и пояснения к реализации

Тема: изучение базовых приёмов работы с классами.

Вариант 5: класс `Three` для беззнаковых целых троичных чисел. Число хранится в динамическом массиве `unsigned char`, каждый элемент содержит цифру 0, 1 или 2. Младшая цифра расположена в элементе с индексом 0.

1. Краткая теория

1.1. Класс, интерфейс и инкапсуляция

Класс объединяет данные и методы для работы с ними. В этой лабораторной интерфейс находится в `Three.hpp`, а реализация — в `Three.cpp`. Такое разделение упрощает чтение и сборку проекта: пользователю класса достаточно знать объявления методов, не изучая их внутреннее устройство.

Поля `digits_` и `size_` объявлены закрытыми (`private`). Поэтому изменить массив извне класса нельзя. Это пример инкапсуляции. Публичные методы позволяют выполнять арифметические операции, сравнения, копирование и преобразование числа в строку.

1.2. Динамическая память

Динамический массив создаётся оператором `new[]`, а освобождается оператором `delete[]` в деструкторе. Размер массива зависит от количества цифр и становится известен во время выполнения программы. `std::vector` для внутреннего хранения не используется, поскольку цель работы — самостоятельно поработать с динамической памятью.

Для каждого объекта выделяется собственный массив. Деструктор вызывается автоматически при уничтожении объекта, поэтому память освобождается без отдельного вызова со стороны пользователя.

1.3. Копирование и перемещение

При обычном копировании указателя оба объекта ссылались бы на один массив. Это называется поверхностным копированием и приводило бы к двойному освобождению памяти. В классе выполняется глубокое копирование: конструктор копирования выделяет новый массив и копирует в него все цифры.

Конструктор перемещения не копирует цифры, а забирает указатель у источника. Он объявлен `noexcept`, поскольку не выделяет память и не выбрасывает исключения. После перемещения источник имеет пустое внутреннее представление: `digits_ == nullptr`, `size_ == 0`. Методы класса рассматривают это состояние как число ноль, поэтому перемещённый объект можно безопасно уничтожать, копировать, сравнивать и использовать в операциях.

Деструктор объявлен `virtual ~Three() noexcept` по образцу `Array`. Спецификатор `noexcept` означает, что деструктор не выбрасывает исключения.

1.4. Представление троичного числа

В обычной записи старшая цифра находится слева, а в массиве — младшая цифра слева. Например:

`102(3) = 1 * 3^2 + 0 * 3^1 + 2 * 3^0 = 11(10)`. Здесь число в скобках обозначает основание системы счисления.

Во внутреннем массиве это число хранится как `[2, 0, 1]`. Такой порядок удобен для сложения и вычитания: обработка начинается с младших разрядов, то есть с нулевого индекса.

Ведущие нули не влияют на значение числа. Метод `normalize()` уменьшает логический размер, пока старшая цифра равна нулю, но оставляет одну цифру для представления нуля. Например, строка `000102` преобразуется в `102`, а `000` — в `0`.

1.5. Сложение и вычитание

Сложение выполняется поразрядно по основанию 3. Для каждой позиции вычисляется сумма двух цифр и переноса:

- цифра результата равна `sum % 3`;
- перенос в следующий разряд равен `sum / 3`.

Пример: `222(3) + 1(3) = 1000(3)`. При сложении последнего разряда возникает дополнительная единица, поэтому результат может быть на один разряд длиннее исходных чисел.

Вычитание также выполняется от младшего разряда к старшему. Если текущая разность отрицательна, к ней прибавляется 3, а из следующего разряда занимается единица. Пример: `1000(3) - 1(3) = 222(3)`.

Перед вычитанием числа сравниваются. Если уменьшаемое меньше вычитаемого, результат был бы отрицательным. Класс работает только с беззнаковыми числами, поэтому в этом случае выбрасывается `std::underflow_error`.

Временная сложность сложения, вычитания и сравнения — `O(n)`, где `n` — количество цифр. Для результата арифметической операции выделяется новый массив, то есть дополнительная память также имеет порядок `O(n)`.

1.6. Сравнение, иммутабельность и исключения

Сначала сравниваются количества разрядов: после удаления ведущих нулей число с большим размером больше. При одинаковых размерах цифры сравниваются от старшей к младшей до первого различия.

Объекты логически иммутабельны: публичных методов изменения цифр нет, а методы операций объявлены `const`. Сложение, вычитание и копирование возвращают новый объект и не меняют операнды. Перегрузка операторов не используется; операции вызываются как обычные методы `add`, `subtract`, `copy`, `equals`, `greater` и `less`.

Конструкторы выбрасывают `std::invalid_argument` при нулевом размере, цифре больше 2, пустой строке или недопустимом символе. Стандартные исключения сообщают об ошибке до начала работы с некорректными данными.

1.7. Модульные тесты

Google Test позволяет автоматически проверять отдельные сценарии. Макрос `EXPECT_EQ` проверяет равенство, `EXPECT_TRUE` — истинность условия, а `EXPECT_THROW` — появление нужного исключения. Тесты не зависят от ручного ввода и после изменения кода запускаются одной командой.

2. Объяснение реализации

2.1. Файл Three.hpp

В заголовочном файле объявлены:

- конструкторы по образцу `Array`: пустой, размер и цифра заполнения, `initializer_list`, строка, копирование и перемещение;
- виртуальный деструктор `noexcept`;
- арифметические методы и методы сравнения;
- вспомогательный метод `toString()` для вывода и проверки результатов;
- закрытые поля `unsigned char* digits_` и `std::size_t size_`.

Метод `logicalSize()` возвращает логическое количество разрядов, включая представление перемещённого объекта как нуля. Метод `digitAt()` возвращает цифру по индексу или ноль, если разряд отсутствует. Благодаря этому операции корректно работают с числами разной длины и с перемещёнными объектами.

2.2. Файл Three.cpp

Конструктор по умолчанию создаёт ноль через конструктор `Three(1, 0)`. Конструктор размера и заполнения проверяет аргументы, выделяет массив и заполняет его указанной цифрой. Конструкторы из списка и строки сначала проверяют входные данные, а затем разворачивают порядок цифр для внутреннего хранения.

Конструктор копирования создаёт независимый массив. Конструктор перемещения только копирует значения указателя и размера, после чего обнуляет источник. Деструктор вызывает `delete[] digits_`; удаление нулевого указателя допустимо.

Метод `add()` создаёт результат с запасом на перенос и проходит по разрядам в одном цикле. Метод `subtract()` проверяет знак результата через сравнение, затем выполняет вычитание с заимствованием. Оба метода вызывают `normalize()` перед возвратом.

Метод `copy()` вызывает конструктор копирования. Метод `equals()` проверяет все цифры, `greater()` реализует сравнение по длине и старшим разрядам, а `less()` использует результаты равенства и сравнения больше. `toString()` разворачивает массив в привычную запись числа.

2.3. Остальные файлы

- `main.cpp` демонстрирует создание чисел, сложение с переносом, вычитание с заимствованием, сравнение, копирование и обработку исключения.

- `test_three.cpp` содержит 21 тест. Проверяются конструкторы, глубокое копирование, перемещение, перенос, заимствование, сравнения, ведущие нули, исключения и неизменность аргументов. Отдельные `static_assert` проверяют `noexcept` перемещения, а также виртуальный и безопасный относительно исключений деструктор.
- `CMakeLists.txt` создаёт библиотеку `three`, программу `three_demo` и тестовую программу `three_tests`. Если Google Test не установлен, CMake загружает его через `FetchContent`.

3. Порядок выполнения работы

1. Изучил требования варианта 5 и конструкторы образца `Array`.
2. Определил внутреннее представление числа: динамический массив `unsigned char`, младший разряд по индексу 0.
3. Разделил класс на интерфейс `Three.hpp` и реализацию `Three.cpp`.
4. Реализовал конструкторы, проверку входных данных и удаление ведущих нулей.
5. Реализовал глубокое копирование, перемещение без выделения памяти и освобождение массива в деструкторе.
6. Написал поразрядные алгоритмы сложения с переносом и вычитания с заимствованием.
7. Добавил копирование методом, сравнения и преобразование числа в строку.
8. Сделал операции неизменяющими исходные объекты и добавил исключение при отрицательной разности.
9. Написал демонстрационную программу `main.cpp`.
10. Подготовил Google Test-тесты для нормальных и ошибочных сценариев.
11. Настроил сборку в `CMakeLists.txt`, собрал проект и запустил тесты. По результатам первого запуска исправил размер массива результата арифметических операций и повторил проверку.
12. Привёл сигнатуры конструкторов и деструктора к образцу, добавил проверки `noexcept` и повторно собрал проект.
13. Подготовил этот отчёт и его PDF-версию.

4. Сборка и запуск

Нужны CMake, компилятор C++14 или новее и доступ к интернету при первой загрузке Google Test.

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

Запуск демонстрации для одноконфигурационного генератора:

```
./build/three_demo
```

Для многоконфигурационного генератора Visual Studio исполняемый файл обычно находится в `build/Debug/three_demo.exe`, а тесты запускаются командой:

```
ctest --test-dir build -C Debug --output-on-failure
```

5. Примеры результатов

Демонстрационная программа использует числа `222(3)` и `1(3)`:

- сумма: `1000(3)`;
- разность: `221(3)`;
- `222(3) > 1(3)` — истина;
- равенство `222(3)` и `1(3)` — ложь;
- копия `222(3)` равна `222(3)`;
- попытка вычислить `1(3) - 222(3)` вызывает ожидаемое исключение.

Дополнительные наборы для устной проверки:

- `2(3) + 2(3) = 11(3)`;
- `120(3) + 12(3) = 202(3)`;
- `120(3) - 12(3) = 101(3)`;
- `201(3) - 2(3) = 122(3)`;
- `000102(3)` после нормализации равно `102(3)`.

6. Загрузка на GitHub

После финальной проверки нужно создать пустой репозиторий на GitHub и отправить в него исходники, тесты, CMake-файл и отчёт. Локально это обычно выполняется так:

```
git init
git add Three.hpp Three.cpp main.cpp test_three.cpp CMakeLists.txt THEORY.md THEORY.pdf
git commit -m "Add Three class lab with tests and report"
git branch -M main
git remote add origin URL_репозитория
git push -u origin main
```

Сборочный каталог `build` и временные инструменты в репозиторий добавлять не нужно.

Заключение

В работе реализован класс беззнаковых троичных чисел на динамическом массиве без использования `std::vector`. Класс поддерживает безопасное владение памятью, глубокое копирование, перемещение, неизменяемые арифметические операции, сравнения, проверку входных данных и исключения. Корректность реализации подтверждается автоматическими тестами и демонстрационной программой.